

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL – UNIDAD III

ASIGNATURA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, wcedenoa2@uteq.edu.ec

CORDOVA CARREÑO MAYRA LUCILA, CI: 0750286775, mcordovac2@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

EQUIPO:

B

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO “B”

SISTEMA PFC:

MundiPets (Andy Mendoza – Titular PFC)

REPOSITORIO GITHUB:

https://github.com/andymendozap03/IngReq_EquipoB_PEU3_Semana10.git

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Introducción	3
2.	Fundamento teórico	4
2.1.	Marco Normativo Aplicado a la Especificación de Requisitos	4
2.1.1.	¿Qué es un marco normativo?	4
2.1.2.	¿Por qué se utilizan normas en Ingeniería de Software?	4
2.1.3.	Norma IEEE/ISO/IEC 29148:2018	4
2.1.4.	Otras normas relacionadas	5
2.1.4a.	ISO/IEC 25010	5
2.1.4b.	ISO/IEC/IEEE 15288	5
2.1.4c.	ISO/IEC/IEEE 12207	5
2.1.5.	Beneficios de aplicar normas en una ERS	5
2.2.	Documentación de Requisitos	6
2.2.1.	¿Qué es la documentación de requisitos?	6
2.2.2.	Objetivos de la documentación de requisitos	6
2.2.3.	Tipos de requisitos	6
2.2.4.	Requisitos funcionales	7
2.2.5.	Requisitos no funcionales	7
2.2.6.	Características de un buen requisito	7
2.2.7.	Importancia de documentar correctamente los requisitos	7
2.3.	Documento ERS/SRS según IEEE/ISO/IEC 29148:2018	8
2.3.1.	¿Qué es un documento ERS o SRS?	8
2.3.2.	Objetivo del documento ERS/SRS	8
2.3.3.	Estructura general según IEEE/ISO/IEC 29148:2018	8
2.3.4.	Secciones principales de una ERS/SRS	9
2.3.4a.	Introducción	9
2.3.4b.	Descripción general	9
2.3.4c.	Requisitos específicos	9
2.3.4d.	Anexos o información complementaria	9
2.3.5.	Beneficios de utilizar este estándar	9
2.4.	Modelos de Datos: Perspectiva de Datos	10
2.4.1.	Concepto y finalidad de los modelos de datos	10
2.4.2.	Entidades, atributos, identificadores y relaciones	10
2.4.3.	Cardinalidad, restricciones e integridad de los datos	10
2.4.4.	Diagrama Entidad-Relación	10
2.4.5.	Diccionario de datos	11
2.4.6.	Importancia de la perspectiva de datos en la Ingeniería de Requisitos	11
2.5.	Modelos Funcionales: Perspectiva Funcional	11
2.5.1.	Concepto y propósito de los modelos funcionales	11
2.5.2.	Requisitos funcionales y funciones del sistema	11
2.5.3.	Casos de uso	11
2.5.4.	Diagramas de actividades	12
2.5.5.	Diagramas de flujo de datos	12
2.5.6.	Aporte de la perspectiva funcional a la especificación de requisitos	12
2.6.	Modelos de Comportamiento: Perspectiva de Comportamiento	12
2.6.1.	Concepto y propósito de los modelos de comportamiento	12
2.6.2.	Comportamiento dinámico del sistema	13
2.6.3.	Diagramas de estados	13
2.6.4.	Diagramas de secuencia	13
2.6.5.	Diagramas de comunicación	13
2.6.6.	Importancia de la perspectiva de comportamiento en la validación de requisitos	13
2.7.	Interrelaciones entre las Tres Perspectivas	13
2.7.1.	Relación entre datos, funciones y comportamiento	13
2.7.2.	Trazabilidad entre los modelos	14
2.7.3.	Consistencia entre las perspectivas	14
2.7.4.	Ejemplo integrado	14

2.7.5.	Importancia de utilizar las tres perspectivas conjuntamente	14
3.	Revisión y refinamiento del SRS parcial	15
3.1.	Registro de defectos del SRS parcial	15
3.2.	Corrección de requisitos con sintaxis EARS	15
3.3.	Tabla de atributos de requisitos (RF y RNF)	15
4.	Modelo de datos: entidad-relación y clases UML	16
4.1.	Identificación de entidades y atributos	16
4.2.	Relaciones y cardinalidad	16
4.3.	Diagrama Entidad-Relación (notación Crow's Foot)	16
4.3.1.	Verificación del cumplimiento de la Tercera Forma Normal (3FN)	16
4.4.	Diagrama de clases UML	17
4.4.1.	Diagrama de Clases Conceptual	17
4.4.2.	Justificación de la Refinación Estructural	17
4.4.3.	Diagrama de Clases Refinado	17
5.	Modelo funcional: DFD y diagrama de actividades UML	18
5.1.	Diagrama de Contexto (DFD Nivel 0)	18
5.2.	DFD Nivel 1	18
5.3.	DFD Esencial	19
5.4.	Diagrama de actividades UML	19
6.	Modelo de comportamiento: estados y secuencia	20
6.1.	Selección de la entidad y ciclo de vida	20
6.2.	Diagrama de máquina de estados UML	21
6.3.	Diagrama de secuencia UML	22
7.	Matriz de trazabilidad y tabla comparativa	23
7.1.	Matriz de trazabilidad	23
7.2.	Identificación de gaps y gold plating	23
7.3.	Verificaciones cruzadas entre las tres perspectivas	23
7.4.	Tabla comparativa de los tres tipos de modelos	23
7.5.	Conclusiones sobre la complementariedad de las tres perspectivas	23
8.	Conclusiones	24
9.	Referencias	25
Anexo A	— Tabla de atributos completa de todos los RF y RNF	26
Anexo B	— Registro de defectos del SRS con correcciones aplicadas	26
Anexo C	— Exportaciones originales de todos los diagramas (alta resolución)	27
Anexo D	— Referencias IEEE y declaración de uso de Inteligencia Artificial	28
Anexo E	— Repositorio en GitHub	29

1. Introducción

La Especificación de Requisitos de Software (ERS o SRS) constituye uno de los documentos más importantes dentro del proceso de desarrollo de software, ya que permite definir de manera clara, organizada y verificable las necesidades que debe satisfacer un sistema antes de su implementación [1, 2]. Una especificación correctamente elaborada establece un lenguaje común entre clientes, usuarios, analistas y desarrolladores, reduciendo ambigüedades y facilitando la planificación, el diseño, la construcción, las pruebas y el mantenimiento del producto de software [3-5]. Esta base terminológica unificada es indispensable para alinear con las expectativas reales de los usuarios finales [6].

La calidad de una especificación de requisitos resulta fundamental para el éxito de un proyecto, debido a que los errores o inconsistencias detectados en etapas posteriores suelen representar un mayor costo en el ciclo de vida del sistema [7, 8]. Por esta razón, estándares internacionales como la norma IEEE/ISO/IEC 29148:2018 proporcionan lineamientos para la documentación, organización y validación de los requisitos, promoviendo características como la completitud, consistencia, trazabilidad y verificabilidad de la información documentada [1, 9]. Asimismo, la definición de estas se apoya en modelos de calidad internacionalmente reconocidos que dictan los atributos de calidad y fiabilidad esperados [10, 11].

En este contexto, la presente práctica experimental desarrolla la especificación de requisitos del sistema MundiPets, una plataforma diseñada para fomentar la cruce responsable de mascotas, facilitar la gestión de adopciones responsables y fortalecer la interacción entre propietarios, posibles adoptantes, médicos veterinarios y refugios de animales. El sistema busca centralizar la información relacionada con las mascotas, permitiendo administrar su historial, publicar procesos de adopción, registrar información veterinaria y brindar herramientas que contribuyan al bienestar y la tenencia responsable de los animales.

Como parte del proceso de especificación, se realiza el refinamiento y organización de los requisitos del sistema aplicando criterios de calidad establecidos en la normativa vigente. Además, se desarrollan modelos que representan el sistema desde tres perspectivas complementarias ampliamente difundidas en las mejores prácticas de la ingeniería de software [2, 6]:

- **La perspectiva de datos:** Diseñada para capturar la estructura estática de la información, se representa mediante el modelo entidad-relación y el diagrama de clases de producción [12, 13].
- **La perspectiva funcional:** Enfocada en los servicios y transformaciones operativas, se ejecuta a través de los diagramas de flujo de datos y el diagrama de actividades [14, 15].
- **La perspectiva de comportamiento:** Orientada a mapear las respuestas dinámicas del software en el tiempo, se construye utilizando los diagramas de máquina de estados y de secuencia [14, 15].

Estos modelos permiten representar de manera estructurada la información, los procesos y el comportamiento dinámico del sistema.

Finalmente, se integra una matriz de trazabilidad que relaciona los requisitos con los diferentes modelos elaborados, permitiendo verificar la cobertura de cada requisito e identificar posibles inconsistencias o elementos que requieran ajustes [1, 2]. De esta manera, el documento presenta una especificación de requisitos más completa, coherente y alineada con las buenas prácticas de la ingeniería de requisitos, constituyendo una base sólida para las siguientes etapas del ciclo de vida del desarrollo de software y contribuyendo a la construcción de una solución que responda adecuadamente a las necesidades planteadas para el sistema MundiPets [8, 16].

2. Fundamento teórico

2.1. Marco Normativo Aplicado a la Especificación de Requisitos

2.1.1. ¿Qué es un marco normativo?

Un marco normativo es el conjunto de normas, estándares, lineamientos, modelos y buenas prácticas que orientan la realización de actividades dentro de un área profesional o técnica [1, 2, 16]. En el ámbito de la Ingeniería de Software, el marco normativo establece criterios formales para planificar, desarrollar, documentar, verificar y mantener productos y procesos de software, con el propósito de asegurar uniformidad, calidad, consistencia, trazabilidad y control durante el ciclo de vida del sistema [1, 2, 16].

Aplicado a la especificación de requisitos, un marco normativo define cómo deben obtenerse, analizarse, documentarse, validarse y mantenerse los requisitos del sistema o del software [3, 12]. No se trata únicamente de un formato documental, sino de un conjunto de principios que ayudan a convertir necesidades de negocio, expectativas de usuarios, restricciones del entorno y objetivos del proyecto en requisitos claros, completos, verificables y gestionables [3, 12].

La existencia de un marco normativo es especialmente importante porque los requisitos constituyen la base sobre la que se diseñará, implementará, probará y mantendrá el software [1, 8]. Por ello, si los requisitos son ambiguos, incompletos o contradictorios, el problema no queda aislado en la etapa de análisis, sino que se propaga al resto del proyecto [1, 8]. En consecuencia, el marco normativo funciona como una guía para reducir la ambigüedad, fortalecer la calidad documental y mejorar la comunicación entre stakeholders [1, 8].

2.1.2. ¿Por qué se utilizan normas en Ingeniería de Software?

Las normas se utilizan en Ingeniería de Software porque permiten estandarizar la forma de trabajar, reducir la improvisación y asegurar que los procesos y artefactos del proyecto se desarrollen bajo criterios de calidad reconocidos [1, 16]. En el caso particular de la ingeniería de requisitos, las normas son fundamentales porque los requisitos constituyen el punto de partida del desarrollo y condicionan todas las etapas posteriores del ciclo de vida del software [1, 16].

Uno de los principales motivos para utilizar normas es la unificación de criterios [3, 12]. Cuando un equipo trabaja con estándares comunes, existe una terminología compartida para hablar de requisitos, atributos de calidad, restricciones, trazabilidad, validación y documentación [3, 12]. Esto facilita la comunicación entre clientes, analistas, desarrolladores, testers, responsables de calidad y patrocinadores del proyecto [3, 12]. Del mismo modo, las normas ayudan a reducir interpretaciones ambiguas, ya que proporcionan lineamientos sobre cómo redactar y estructurar la información para que resulte comprensible y verificable [3, 12].

Otro motivo importante es la mejora de la calidad de los requisitos y de la documentación [8, 13]. Las normas promueven que los requisitos sean claros, completos, consistentes, verificables y trazables, atributos que la literatura reciente sigue identificando como esenciales para prevenir defectos en etapas posteriores [8, 13]. La investigación contemporánea sobre calidad de requisitos confirma que los problemas en la especificación siguen siendo una causa importante de fallos, retrabajo y dificultades en validación, lo que refuerza la necesidad de utilizar marcos normativos y criterios sistemáticos de calidad [8, 13].

Además, las normas facilitan la trazabilidad y la gestión del cambio [1, 3]. En proyectos reales, los requisitos evolucionan, se refinan y se relacionan con decisiones de diseño, casos de prueba, restricciones técnicas y necesidades del negocio [1, 3]. Un marco normativo ayuda a documentar estas relaciones de manera controlada, de modo que sea posible identificar el origen de un requisito, seguir su evolución y analizar el impacto de cualquier modificación [1, 3]. Esta capacidad es clave para mantener la coherencia del proyecto a lo largo del tiempo [1, 3].

En síntesis, las normas se utilizan en Ingeniería de Software porque aportan orden, calidad, trazabilidad, capacidad de verificación y coherencia metodológica [2, 12]. Su aplicación no debe verse como un requisito burocrático, sino como una práctica que mejora la comunicación, reduce errores tempranos y fortalece la base documental sobre la cual se construye el software [2, 12].

2.1.3. Norma IEEE/ISO/IEC 29148:2018

La ISO/IEC/IEEE 29148:2018 es la norma de referencia más importante para la ingeniería de requisitos [1]. Su propósito es proporcionar un marco para la elicitación, análisis, especificación, validación y gestión de requisitos, así como lineamientos para la documentación de requisitos de sistemas y de software [1]. Esta norma integra perspectivas de la ingeniería de sistemas y de software, y conecta la especificación de requisitos con el ciclo de vida completo del sistema [1].

Uno de sus aportes centrales es que define la ingeniería de requisitos como un proceso disciplinado, no como

una actividad aislada de redacción [1]. Desde este enfoque, los requisitos deben ser identificados, analizados, negociados, documentados, validados y mantenidos de forma controlada [1]. La norma enfatiza que un requisito no es simplemente una “idea” o una “necesidad expresada”, sino una especificación que debe contar con la calidad suficiente para ser comprendida, implementada y verificada objetivamente [1].

La 29148:2018 también establece criterios de calidad para los requisitos [1]. En términos generales, promueve que los requisitos sean correctos, completos, consistentes, no ambiguos, verificables, trazables, modificables y necesarios [1]. Estos atributos son esenciales porque convierten al requisito en una unidad de información útil para el desarrollo y la evaluación del sistema [1]. La literatura reciente sobre calidad de requisitos confirma la vigencia de estos atributos y destaca que la claridad, la consistencia y la verificabilidad continúan siendo dimensiones fundamentales en la evaluación de la calidad de una especificación [8, 13].

Otro aspecto clave de la norma es su aporte a la documentación de requisitos [1]. La 29148 no se limita a definir atributos abstractos, sino que también ofrece orientación para estructurar documentos como la ERS/SRS, diferenciar niveles de requisitos y separar necesidades de stakeholders, requisitos del sistema, requisitos del software, restricciones e interfaces [1]. Gracias a ello, la norma no solo apoya la calidad del requisito individual, sino también la calidad del documento que agrupa esos requisitos [1].

Finalmente, la ISO/IEC/IEEE 29148:2018 enfatiza la trazabilidad bidireccional, es decir, la posibilidad de rastrear cada requisito desde su origen hasta su implementación y prueba, y viceversa [1]. Esta capacidad es fundamental para justificar la existencia de cada requisito, verificar su cumplimiento y gestionar cambios sin perder coherencia en el proyecto [1]. Por esta razón, la 29148:2018 se considera el estándar principal para la elaboración y gestión de una ERS/SRS de calidad [1].

2.1.4. Otras normas relacionadas

Aunque la ISO/IEC/IEEE 29148:2018 es la norma central para la ingeniería de requisitos, existen otros estándares que complementan su aplicación y enriquecen la especificación de requisitos desde perspectivas como la calidad del producto, el ciclo de vida del sistema y el ciclo de vida del software [1, 8].

2.1.4a. ISO/IEC 25010

La ISO/IEC 25010 define un modelo de calidad del producto software y de la calidad en uso [11]. Esta norma es especialmente relevante para la documentación de requisitos porque muchas de sus características se traducen directamente en requisitos no funcionales [11]. Entre las características más importantes del modelo se encuentran la adecuación funcional, eficiencia del desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad [11].

La importancia de esta norma radica en que ayuda a transformar expectativas generales de calidad en requisitos específicos y verificables [3, 12]. Por ejemplo, si un sistema debe ser seguro, usable, eficiente o mantenible, esas cualidades no deberían quedar como ideas abstractas, sino expresarse como requisitos concretos que puedan evaluarse [3, 12]. La investigación reciente sobre *quality requirements* respalda esta necesidad, señalando que los requisitos no funcionales continúan siendo difíciles de documentar y evaluar en la práctica, precisamente por su naturaleza transversal y por la falta de tratamiento sistemático en muchos proyectos [17].

2.1.4b. ISO/IEC/IEEE 15288

La ISO/IEC/IEEE 15288 establece procesos del ciclo de vida de sistemas [8]. Su relación con la especificación de requisitos es directa porque sitúa la definición de necesidades y requisitos dentro de un proceso mayor que incluye la concepción del sistema, la arquitectura, la integración, la validación, la operación y el mantenimiento [8]. Aunque su alcance es más amplio que el del software, esta norma es útil porque recuerda que los requisitos del software no deben analizarse de forma aislada, sino como parte de una solución sistémica más grande [8].

2.1.4c. ISO/IEC/IEEE 12207

La ISO/IEC/IEEE 12207 es una norma clásica sobre procesos del ciclo de vida del software. Aunque no es el estándar principal para especificar requisitos, se relaciona con este tema porque ubica la ingeniería de requisitos dentro del conjunto de procesos de desarrollo, mantenimiento y soporte del software. Su mención es pertinente cuando se desea contextualizar la ERS dentro de un marco más amplio de procesos de ingeniería de software.

2.1.5. Beneficios de aplicar normas en una ERS

La aplicación de normas en la elaboración de una ERS (Especificación de Requisitos de Software) ofrece beneficios metodológicos, técnicos y organizativos [8]. En primer lugar, proporciona una estructura uniforme para organizar el documento, lo que mejora su legibilidad, facilita la revisión y permite mantener la información de forma más controlada [8]. Una ERS basada en estándares evita documentos desordenados, ambiguos o excesivamente informales [8].

En segundo lugar, las normas favorecen la calidad de los requisitos, ya que promueven atributos como claridad, completitud, consistencia, verificabilidad y trazabilidad [8]. Esto mejora la utilidad del documento como base para diseño, implementación, pruebas y mantenimiento [8]. De hecho, la literatura reciente sobre especificación en la práctica muestra que la estructura del documento, el uso de plantillas y la claridad de los artefactos siguen siendo factores relevantes en la forma en que las organizaciones gestionan sus requisitos [18].

Un tercer beneficio es la facilidad para validar y probar [1, 3]. Cuando los requisitos están redactados de manera precisa y verificable, es más sencillo derivar criterios de aceptación, casos de prueba y mecanismos de validación [1, 3]. Asimismo, la normalización mejora la trazabilidad y el control de cambios, ya que permite relacionar requisitos con su origen, con otros artefactos del proyecto y con sus evidencias de verificación [1, 3].

Finalmente, aplicar normas en una ERS contribuye a la calidad del producto final, porque un sistema construido sobre requisitos bien definidos tiene más probabilidades de satisfacer necesidades reales del usuario y cumplir con atributos de calidad esperados [8]. En consecuencia, el marco normativo no solo mejora el documento, sino también el proceso de desarrollo y el resultado del proyecto [8].

2.2. Documentación de Requisitos

2.2.1. ¿Qué es la documentación de requisitos?

La documentación de requisitos es el proceso mediante el cual se registran, organizan, describen y comunican las necesidades, restricciones, funciones y atributos de calidad que debe cumplir un sistema de software [1, 3]. Su resultado habitual es un documento formal, como una ERS/SRS, donde se consolidan los requisitos acordados entre las partes interesadas y se define la base del desarrollo del sistema [1, 3].

Documentar requisitos no consiste únicamente en escribir una lista de funcionalidades [12]. Implica transformar necesidades del negocio, objetivos del proyecto, restricciones del entorno y expectativas de los usuarios en especificaciones claras, consistentes y verificables, de manera que puedan servir como insumo para el análisis, diseño, implementación, pruebas y mantenimiento del software [12]. Por esta razón, la documentación de requisitos es uno de los productos más importantes de la ingeniería de requisitos y uno de los principales mecanismos para alinear la visión del cliente con la del equipo de desarrollo [12].

La importancia de documentar requisitos también se confirma en la práctica industrial [18]. Estudios recientes sobre *requirements specification* muestran que la especificación sigue siendo una actividad central en los proyectos de software, y que las organizaciones continúan enfrentando desafíos relacionados con la estructura del documento, el uso de plantillas, la granularidad de los requisitos y la calidad del contenido [18].

2.2.2. Objetivos de la documentación de requisitos

La documentación de requisitos tiene varios objetivos esenciales [1]. El primero es capturar de forma precisa las necesidades de los stakeholders, evitando que se pierdan, se deformen o queden sujetas a interpretaciones ambiguas [1]. Al registrar estas necesidades de manera formal, el equipo de desarrollo cuenta con una base estable para planificar el sistema y construirlo con mayor coherencia [1].

Un segundo objetivo es establecer una base común de entendimiento entre clientes, usuarios, analistas, desarrolladores, testers y responsables del proyecto [3, 12]. La documentación funciona como un punto de acuerdo que permite a todos consultar la misma referencia y compartir una visión común del producto [3, 12]. Esto reduce malentendidos y facilita la toma de decisiones a lo largo del ciclo de vida del software [3, 12].

También tiene como propósito delimitar el alcance del sistema [1]. Una buena documentación ayuda a definir qué está incluido y qué no en el producto a desarrollar, lo que reduce el riesgo de crecimiento descontrolado del alcance, solicitudes informales de cambio o conflictos sobre expectativas del cliente [1]. Del mismo modo, sirve como base para el diseño, la implementación y las pruebas, ya que los equipos técnicos necesitan una especificación clara para construir el sistema y comprobar si realmente cumple con lo solicitado [1].

Finalmente, la documentación de requisitos apoya la trazabilidad, la validación y la gestión de cambios [1]. Permite relacionar requisitos con su origen, registrar versiones, justificar modificaciones y analizar el impacto de cambios sobre otras partes del sistema [1]. Por ello, su función no termina en la etapa de análisis, sino que acompaña al software durante todo su ciclo de vida [1].

2.2.3. Tipos de requisitos

Existen diferentes formas de clasificar los requisitos; sin embargo, una de las más importantes distingue entre requisitos funcionales y requisitos no funcionales [3, 12]. Esta clasificación es ampliamente utilizada porque separa lo que el sistema debe hacer de las condiciones o atributos con los que debe hacerlo [3, 12].

2.2.4. Requisitos funcionales

Los requisitos funcionales describen los servicios, procesos, comportamientos o acciones que el sistema debe ejecutar para satisfacer una necesidad del usuario o del negocio [1]. En otras palabras, expresan las funciones que el software debe ofrecer y responden a preguntas como: qué operaciones realizará el sistema, qué información procesará, qué resultados producirá y cómo reaccionará ante determinadas entradas o eventos [1].

Ejemplos de requisitos funcionales son afirmaciones como las siguientes: el sistema debe permitir registrar nuevos usuarios; el sistema debe generar reportes mensuales de ventas; el sistema debe validar credenciales de acceso; o el sistema debe permitir consultar el historial de pedidos de un cliente [1]. Todos estos enunciados describen qué hace el sistema desde una perspectiva funcional [1].

Los requisitos funcionales constituyen el núcleo visible de muchas especificaciones, ya que suelen representar aquello que el cliente y el usuario identifican con mayor facilidad [3]. Sin embargo, por sí solos no son suficientes para garantizar la calidad del sistema, ya que también es necesario especificar las restricciones y atributos de calidad que condicionan la forma en que dichas funciones deben ejecutarse [3].

2.2.5. Requisitos no funcionales

Los requisitos no funcionales describen cómo debe comportarse el sistema o qué restricciones debe cumplir [3, 12]. No se centran en una función específica, sino en atributos de calidad, limitaciones técnicas, condiciones operativas o políticas que afectan al sistema en su conjunto o a una parte de él [3, 12]. Entre los aspectos que suelen abarcar se encuentran el rendimiento, la seguridad, la disponibilidad, la usabilidad, la compatibilidad, la mantenibilidad y la portabilidad [11].

Ejemplos de requisitos no funcionales son afirmaciones como: el sistema debe responder en menos de tres segundos; la información sensible debe almacenarse cifrada; la plataforma debe estar disponible al menos el 99,5 % del tiempo; o la interfaz debe ser usable por usuarios sin experiencia técnica [11]. En estos casos, el requisito no describe una función específica, sino una condición de calidad o restricción que afecta el comportamiento del software [11].

La ISO/IEC 25010 es una referencia clave para este tipo de requisitos, ya que proporciona un modelo de calidad del producto software que puede utilizarse para identificar y organizar atributos de calidad en la especificación [11]. Además, la investigación reciente sobre *quality requirements* demuestra que los requisitos no funcionales siguen siendo uno de los temas más complejos de documentar y evaluar, lo que hace aún más importante el uso de modelos normativos para estructurarlos adecuadamente [17].

2.2.6. Características de un buen requisito

La calidad de una especificación depende en gran medida de la calidad individual de sus requisitos [1, 3]. Un buen requisito no solo debe expresar una necesidad, sino hacerlo de forma que pueda comprenderse, implementarse, verificarse y mantenerse [1, 3]. La ISO/IEC/IEEE 29148 y la literatura especializada en ingeniería de requisitos coinciden en que un requisito de calidad debe poseer varias características fundamentales [1, 3].

En primer lugar, un requisito debe ser claro, es decir, redactado con un lenguaje preciso, comprensible y libre de ambigüedades [1]. También debe ser completo, de manera que contenga la información necesaria para entender qué se solicita, en qué contexto aplica y bajo qué condiciones debe cumplirse [1]. Además, debe ser correcto, lo que implica que represente fielmente una necesidad real del usuario, del negocio o del sistema [1].

Otra característica es la consistencia [1]. Un requisito no debe contradecir a otros requisitos ni entrar en conflicto con reglas de negocio, restricciones del sistema o decisiones previamente acordadas [1]. Igualmente, debe ser verificable, es decir, debe poder comprobarse objetivamente mediante revisión, prueba, análisis o demostración [12]. Si un requisito no puede verificarse, su cumplimiento resulta difícil de evaluar [12].

También es fundamental la trazabilidad, ya que permite relacionar el requisito con su origen y con los artefactos derivados, como diseño, implementación, pruebas o documentación de soporte [3]. Del mismo modo, un requisito debe ser modificable, de forma que pueda actualizarse sin generar confusión ni afectar innecesariamente al resto del documento [3]. Finalmente, debe ser factible o viable y necesario, es decir, implementable dentro de las restricciones del proyecto y justificado por una necesidad real [3].

La investigación reciente sobre calidad de requisitos respalda estas características y subraya que la claridad, la consistencia, la ausencia de ambigüedad y la verificabilidad siguen siendo ejes centrales para evaluar la calidad de una especificación y prevenir defectos en el desarrollo posterior [8, 13].

2.2.7. Importancia de documentar correctamente los requisitos

La correcta documentación de requisitos es una actividad crítica porque los requisitos constituyen la base de todo el desarrollo del software [3, 12]. Si están mal definidos, incompletos o redactados de forma ambigua, el

problema se traslada al diseño, a la implementación, a las pruebas y al mantenimiento, generando retrabajo, sobrecostos y fallos en el producto final [3, 12].

Una documentación adecuada reduce malentendidos entre cliente y equipo de desarrollo, ya que establece una referencia común sobre el comportamiento esperado del sistema [1]. También ayuda a definir el alcance del proyecto, evitando que se incorporen funciones no acordadas o que se omitan necesidades importantes [1]. Además, funciona como un acuerdo técnico y funcional entre las partes, al registrar formalmente lo que el sistema debe hacer y bajo qué condiciones debe hacerlo [1].

Otro aspecto relevante es que la documentación de requisitos guía el diseño, la implementación y la validación del sistema [2]. Los desarrolladores necesitan requisitos claros para construir soluciones adecuadas, y los testers requieren requisitos verificables para elaborar casos de prueba y criterios de aceptación [2]. Del mismo modo, la documentación facilita el mantenimiento y la evolución del software, ya que permite comprender qué se definió originalmente, por qué se hizo y cómo podrían afectar los cambios futuros [2].

La evidencia reciente también respalda esta importancia [8, 17, 18]. Estudios sobre *quality requirements*, *quality of requirements* y *requirements specification* muestran que la calidad de la especificación sigue siendo un factor determinante para el éxito del desarrollo, y que los problemas de redacción, estructura o falta de claridad continuúan afectando la práctica profesional [8, 17, 18].

En definitiva, documentar correctamente los requisitos no es una actividad administrativa secundaria, sino una práctica central de la ingeniería de software, porque transforma necesidades y expectativas en una base formal para construir un sistema de calidad [8, 17, 18].

2.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

2.3.1. ¿Qué es un documento ERS o SRS?

La ERS (Especificación de Requisitos de Software) o SRS (*Software Requirements Specification*) es el documento formal en el que se describen de manera estructurada los requisitos funcionales, no funcionales, restricciones, interfaces, supuestos y demás condiciones que debe cumplir un sistema de software [1]. Su finalidad es consolidar en un único artefacto la información necesaria para comprender qué debe hacer el sistema, cómo debe comportarse y bajo qué condiciones debe operar [1].

La ERS/SRS es uno de los productos de trabajo más importantes de la ingeniería de requisitos, porque sirve como base para el diseño, la implementación, las pruebas, la validación, la aceptación y el mantenimiento del sistema [3, 12]. Además, actúa como medio de comunicación entre clientes, usuarios, analistas, desarrolladores y responsables de calidad [3, 12]. Cuando está elaborada con base en una norma como la ISO/IEC/IEEE 29148:2018, adquiere mayor consistencia, trazabilidad y utilidad como instrumento de trabajo a lo largo del ciclo de vida del software [8].

2.3.2. Objetivo del documento ERS/SRS

El objetivo principal de una ERS/SRS es comunicar de manera clara, completa, consistente y verificable los requisitos del software, de modo que todas las partes involucradas compartan una visión común del sistema a construir [1]. No se trata solo de “guardar” requisitos, sino de hacerlo con el nivel de precisión necesario para que el documento pueda servir como base real para el desarrollo y la validación del sistema [1].

Entre sus objetivos específicos se encuentran: documentar formalmente los requisitos del software; servir como base para diseño, desarrollo y pruebas; facilitar la validación del sistema; delimitar el alcance del proyecto; apoyar la trazabilidad y la gestión de cambios; y mejorar la comunicación entre stakeholders [3]. En conjunto, estos objetivos convierten a la ERS en una pieza central de la gobernanza técnica del proyecto [3].

2.3.3. Estructura general según IEEE/ISO/IEC 29148:2018

La IEEE/ISO/IEC 29148:2018 proporciona lineamientos para la especificación de requisitos y propone una estructura general para documentos de requisitos de sistema y de software [1]. Aunque la forma exacta del documento puede adaptarse al tipo de proyecto, al contexto organizacional o al ámbito académico, la norma sugiere una organización lógica del contenido que favorezca la comprensión, la revisión, la mantenibilidad y la trazabilidad de la especificación [1].

En términos generales, una ERS/SRS suele estructurarse en cuatro grandes bloques: Introducción, Descripción general, Requisitos específicos, y Anexos o información complementaria [1]. Esta estructura es relevante porque separa la información de contexto, la visión global del sistema, el detalle de los requisitos y el material de apoyo [1]. De esta manera, el documento no solo presenta requisitos, sino que los organiza de forma que puedan ser entendidos, revisados y utilizados por diferentes tipos de lectores [1].

2.3.4. Secciones principales de una ERS/SRS

2.3.4a. Introducción

La introducción presenta el contexto general del documento y permite al lector comprender para qué se elaboró la especificación, a qué sistema se refiere y cómo está organizada [1]. En esta sección suelen incluirse el propósito del documento, el alcance del sistema, las definiciones, acrónimos y abreviaturas, las referencias y una visión general de la estructura del documento [1]. La función de esta sección es contextualizar el resto de la especificación [1]. Sin una introducción adecuada, el lector puede comprender requisitos individuales, pero no necesariamente el propósito global del sistema ni la lógica con la que fue construida la ERS [1].

2.3.4b. Descripción general

La descripción general ofrece una visión de alto nivel del sistema antes de entrar al detalle de los requisitos específicos [1]. Su objetivo es situar el producto dentro de su contexto operativo, organizacional y técnico [1, 3]. En esta sección suelen incluirse la perspectiva del producto, las funciones generales del sistema, las características de los usuarios, el entorno operativo, las restricciones y los supuestos o dependencias relevantes [1, 3]. Esta parte del documento es importante porque ayuda a comprender el contexto del software, su relación con otros sistemas y las condiciones bajo las cuales deberá operar [1, 3]. Además, ofrece información previa que da sentido a los requisitos detallados que aparecerán después [1, 3].

2.3.4c. Requisitos específicos

La sección de requisitos específicos constituye el núcleo de la ERS/SRS [1]. Aquí se detallan, de manera organizada y verificable, los requisitos que el sistema debe cumplir [1]. Dependiendo del proyecto, esta sección puede estructurarse por módulos, por casos de uso, por subsistemas o por tipo de requisito [1].

Normalmente incluye requisitos funcionales, requisitos no funcionales, requisitos de interfaces externas, requisitos de datos, reglas de negocio y restricciones técnicas o normativas [1]. Cada requisito debe redactarse de forma clara, verificable y trazable, preferiblemente con un identificador único que facilite su revisión, implementación y prueba [1]. Esta sección es la más importante de la ERS porque define con precisión qué debe construirse y en qué condiciones debe hacerlo el sistema [1]. La relevancia práctica de esta sección se refleja en estudios recientes sobre *requirements specification*, que muestran que la forma en que las organizaciones estructuran sus especificaciones, utilizan plantillas y documentan sus requisitos influye directamente en la calidad de la comunicación, la consistencia del trabajo y la capacidad de gestionar el producto en etapas posteriores [18].

2.3.4d. Anexos o información complementaria

Los anexos reúnen información de apoyo que, aunque no forma parte del cuerpo principal de requisitos, ayuda a comprender mejor el sistema o a complementar la especificación [1]. Entre los elementos que pueden incluirse se encuentran glosarios, diagramas, prototipos, tablas de trazabilidad, modelos de datos, reglas de negocio ampliadas, catálogos de interfaces o documentos de apoyo [1]. Su utilidad radica en que permiten enriquecer la especificación sin sobrecargar las secciones principales, además de facilitar la comunicación con usuarios, desarrolladores y evaluadores mediante material visual o técnico complementario [1].

2.3.5. Beneficios de utilizar este estándar

El uso del estándar IEEE/ISO/IEC 29148:2018 para elaborar una ERS/SRS aporta ventajas concretas tanto en la calidad del documento como en la gestión del proyecto de software [1]. En primer lugar, proporciona una estructura uniforme, lo que mejora la organización, la legibilidad y la revisión del documento [1]. En segundo lugar, promueve requisitos de mayor calidad al insistir en atributos como claridad, consistencia, completitud, verificabilidad y trazabilidad [1].

También mejora la comunicación entre las partes interesadas, ya que todos trabajan sobre una base documental y terminológica común [3, 12]. Asimismo, facilita la validación y las pruebas, porque los requisitos bien redactados pueden transformarse con mayor facilidad en criterios de aceptación y casos de prueba [3, 12]. Otro beneficio importante es el apoyo al control de cambios, debido a que una especificación bien organizada permite identificar el impacto de modificaciones y mantener actualizada la documentación [3, 12].

La investigación reciente sobre calidad de requisitos y especificación en la práctica respalda estos beneficios [8, 18]. Por una parte, se ha señalado que la calidad de los requisitos sigue siendo un factor decisivo para prevenir defectos y mejorar el desarrollo posterior; por otra, se ha observado que la estructura del documento, el uso de guías y la disciplina en la especificación continúan siendo variables relevantes en el trabajo real de las organizaciones [8, 18]. En consecuencia, utilizar la ISO/IEC/IEEE 29148:2018 no solo mejora la presentación formal de una ERS, sino que convierte el documento en una herramienta de trabajo sólida para el análisis, la construcción, la validación y la evolución del sistema [1].

2.4. Modelos de Datos: Perspectiva de Datos

2.4.1. Concepto y finalidad de los modelos de datos

Los modelos de datos constituyen una representación conceptual de la información que un sistema de software debe almacenar, procesar y administrar durante su operación [1]. En el contexto de la Ingeniería de Requisitos, estos modelos permiten describir la estructura lógica de los datos antes de iniciar el diseño de la base de datos o el desarrollo del sistema, facilitando la comprensión de los requisitos relacionados con la información [1]. La especificación de requisitos debe identificar claramente los datos que intervienen en cada proceso del sistema, ya que estos forman parte esencial de la documentación que guía las etapas posteriores del ciclo de vida del software [16].

Desde una perspectiva de análisis, los modelos de datos ayudan a transformar las necesidades del usuario en estructuras organizadas que posteriormente serán implementadas mediante tablas, objetos u otros mecanismos de almacenamiento [2]. El modelado de datos reduce las ambigüedades durante la ingeniería de requisitos y facilita la comunicación entre clientes, analistas y desarrolladores, debido a que todos trabajan sobre una misma representación conceptual de la información [2]. Asimismo, el modelado de datos constituye una actividad esencial dentro del análisis del software, ya que permite comprender el dominio del problema antes de definir procesos o diseñar componentes, disminuyendo significativamente la probabilidad de errores durante la implementación [3, 16].

El objetivo principal de un modelo de datos consiste en representar de manera organizada la información que será utilizada por el sistema, estableciendo qué datos existen, cuáles son sus características y cómo se relacionan entre sí [12]. Un modelo correctamente elaborado permite garantizar la integridad, consistencia y disponibilidad de la información durante todo el ciclo de vida del software [12]. Entre los objetivos destacan: identificar la información necesaria, organizar los datos de forma estructurada, reducir redundancias e inconsistencias, facilitar el diseño de bases de datos, mejorar la comunicación entre usuarios y desarrolladores, y servir como base para el diseño lógico y físico del sistema [12]. La correcta definición de la información constituye un elemento fundamental para garantizar la calidad de los productos desarrollados, ya que los datos representan uno de los activos más importantes de cualquier sistema de información [8].

2.4.2. Entidades, atributos, identificadores y relaciones

Todo modelo de datos está conformado por tres elementos fundamentales: entidades, atributos y relaciones [12, 13]. Las entidades representan objetos, personas, lugares, eventos o conceptos del mundo real acerca de los cuales el sistema necesita almacenar información [12, 13]. Cada entidad posee identidad propia y puede ser identificada de manera única dentro del modelo [12, 13]. La correcta identificación de entidades constituye uno de los primeros pasos del análisis de requisitos, ya que permite definir el alcance de la información que administrará el sistema [3].

Los atributos representan las características o propiedades de una entidad [12]. Cada atributo almacena información específica que describe un objeto determinado [12]. Una adecuada definición de atributos permite evitar duplicidad de información y facilita la normalización de la base de datos [12]. Las relaciones describen la asociación existente entre dos o más entidades [12, 13]. Estas asociaciones permiten representar las reglas del negocio y constituyen la base para definir las cardinalidades, fundamentales durante el diseño de bases de datos relacionales [12, 13].

2.4.3. Cardinalidad, restricciones e integridad de los datos

La cardinalidad define el número de instancias de una entidad que pueden participar en una relación [12]. Los tipos más comunes son uno a uno (1:1), uno a muchos (1:N) y muchos a muchos (N:M) [12]. Las restricciones de integridad, como la integridad de entidad y la integridad referencial, garantizan que los datos sean consistentes y válidos [12]. La clave primaria identifica de manera única cada instancia de una entidad, mientras que las claves foráneas establecen los vínculos entre entidades y aseguran que no existan referencias a datos inexistentes [3].

2.4.4. Diagrama Entidad-Relación

El Diagrama Entidad-Relación (DER) es uno de los modelos conceptuales más utilizados para representar gráficamente la estructura de los datos de un sistema [12, 13]. Propuesto por Chen [13], permite representar visualmente las entidades, sus atributos, las relaciones, las cardinalidades y las claves primarias y foráneas [12, 13]. El principal beneficio del DER consiste en facilitar la comunicación entre usuarios y desarrolladores, ya que proporciona una representación sencilla del dominio del problema antes de comenzar la implementación física de la base de datos [2, 12]. Además, el DER permite detectar errores tempranos relacionados con información duplicada, relaciones incorrectas o ausencia de entidades importantes, reduciendo costos durante las etapas

posteriores del desarrollo [2, 12].

2.4.5. Diccionario de datos

El diccionario de datos constituye un documento complementario al DER donde se describe detalladamente cada uno de los elementos que forman parte de la base de datos [3]. Generalmente incluye información como: nombre del atributo, descripción, tipo de dato, longitud, restricciones, valores permitidos, clave primaria, clave foránea y reglas de validación [3]. Este documento facilita la comprensión del sistema, mejora la trazabilidad de los requisitos y sirve como referencia durante las actividades de mantenimiento del software [3]. El diccionario de datos reduce la ambigüedad entre los miembros del equipo y favorece la estandarización de la información utilizada durante el desarrollo [10, 16].

2.4.6. Importancia de la perspectiva de datos en la Ingeniería de Requisitos

Los modelos de datos constituyen uno de los pilares fundamentales de la Ingeniería de Requisitos porque permiten comprender la información que será manipulada por el sistema antes de definir los procesos funcionales o el comportamiento dinámico de la aplicación [3, 10]. Una especificación incompleta de los datos puede ocasionar información redundante, inconsistencias, errores durante el diseño, baja calidad de la base de datos y dificultades de mantenimiento [3, 10]. Por el contrario, un modelo de datos correctamente elaborado facilita la validación de requisitos, mejora la comunicación entre las partes interesadas y proporciona una base sólida para las siguientes etapas del desarrollo [3, 10]. Una adecuada gestión de la información contribuye directamente a características como la mantenibilidad, confiabilidad, integridad y eficiencia del sistema [11].

2.5. Modelos Funcionales: Perspectiva Funcional

2.5.1. Concepto y propósito de los modelos funcionales

Los modelos funcionales constituyen una representación de las funciones, procesos y servicios que un sistema de software debe proporcionar para satisfacer las necesidades de los usuarios [1]. A diferencia de los modelos de datos, que describen la información administrada por el sistema, los modelos funcionales se enfocan en las actividades que el software realiza y en la manera en que los usuarios interactúan con dichas funcionalidades [1]. La especificación de requisitos debe describir claramente las funciones esperadas del sistema, indicando qué acciones debe ejecutar, bajo qué condiciones y cuáles serán los resultados esperados [1].

Durante la Ingeniería de Requisitos, los modelos funcionales permiten transformar las necesidades expresadas por los usuarios en procesos claramente definidos, facilitando la comunicación entre clientes, analistas y desarrolladores [2]. Además, constituyen una herramienta esencial para verificar que todas las funcionalidades solicitadas hayan sido identificadas y documentadas antes de iniciar el diseño o la implementación del software [2, 4]. El modelado funcional representa uno de los principales mecanismos para comprender el comportamiento operativo del sistema, ya que permite identificar las actividades que generan valor para el usuario y la forma en que estas se relacionan con los objetivos del negocio [3, 16].

El principal objetivo de los modelos funcionales consiste en representar de manera organizada todas las funciones que debe realizar un sistema para cumplir con los requisitos establecidos por el cliente [10]. Entre los objetivos más importantes se encuentran: identificar las funciones principales del sistema, describir los procesos de negocio, facilitar la comunicación entre usuarios y desarrolladores, detectar funciones faltantes o redundantes, servir como base para el diseño y las pruebas del software, y mejorar la comprensión de los requisitos funcionales [10]. La correcta elaboración de modelos funcionales permite validar tempranamente el alcance del sistema y reducir el riesgo de desarrollar funcionalidades innecesarias o de omitir procesos importantes [10].

2.5.2. Requisitos funcionales y funciones del sistema

Los requisitos funcionales especifican qué debe hacer el sistema, es decir, describen las interacciones entre el sistema y su entorno [4]. Estos requisitos se derivan de las necesidades de los usuarios y se expresan en términos de servicios, tareas o comportamientos que el software debe ofrecer [10]. La identificación y documentación de estos requisitos es fundamental para establecer el alcance del proyecto y para guiar las fases de diseño e implementación [3].

2.5.3. Casos de uso

Uno de los modelos funcionales más utilizados en Ingeniería de Software son los casos de uso [10]. Un caso de uso describe la interacción entre un actor y el sistema para lograr un objetivo específico, mostrando las acciones que debe ejecutar el software para satisfacer una necesidad del usuario [10]. Cada caso de uso representa un escenario de utilización del sistema y generalmente incluye: nombre, objetivo, actor principal, actores secundarios, precondiciones, flujo principal de eventos, flujos alternativos y postcondiciones [5, 10]. Los

casos de uso permiten comprender fácilmente las funcionalidades del sistema desde el punto de vista del usuario, convirtiéndose en una herramienta fundamental para la validación de requisitos y la comunicación entre todas las partes interesadas [3, 5]. Además, sirven como base para la elaboración de diagramas UML y el diseño de casos de prueba durante las etapas posteriores del desarrollo [3, 5].

2.5.4. Diagramas de actividades

Los diagramas de actividades pertenecen al lenguaje UML [14] y permiten representar gráficamente el flujo de ejecución de un proceso mediante acciones, decisiones, bifurcaciones y sincronizaciones [14]. Su principal finalidad consiste en describir la secuencia lógica de actividades que realiza el sistema para completar una determinada funcionalidad [15]. Estos diagramas están conformados por elementos como nodo inicial, actividades, decisiones, flujos de control, barras de sincronización y nodo final [14]. Una de sus principales ventajas consiste en que permiten visualizar procesos complejos de manera sencilla, facilitando la identificación de cuellos de botella, actividades redundantes y posibles mejoras antes de implementar el sistema [10]. Los diagramas de actividades permiten representar claramente los procesos del negocio y constituyen un complemento importante para los casos de uso, ya que muestran el flujo interno de cada funcionalidad [2, 15].

2.5.5. Diagramas de flujo de datos

En algunas metodologías tradicionales de análisis de sistemas todavía se utilizan los Diagramas de Flujo de Datos (DFD) para representar la circulación de la información dentro del sistema [3]. Aunque UML ha adquirido mayor protagonismo durante los últimos años, los DFD continúan siendo una herramienta útil para comprender cómo viajan los datos entre procesos, entidades externas y almacenes de información [3]. Los elementos principales de un DFD son: procesos, flujos de datos, entidades externas y almacenes de datos [3]. Los DFD permiten representar únicamente el movimiento de la información, sin preocuparse por aspectos relacionados con la implementación tecnológica [3]. Esto facilita la identificación de entradas, salidas y transformaciones de los datos durante la ejecución de los procesos [10].

2.5.6. Aporte de la perspectiva funcional a la especificación de requisitos

La perspectiva funcional constituye uno de los pilares de la especificación de requisitos, ya que permite describir con precisión el comportamiento esperado del sistema desde el punto de vista del usuario [4]. Su utilización facilita la identificación de procesos críticos, mejora la comunicación entre las partes interesadas y reduce el riesgo de desarrollar funcionalidades que no aporten valor al negocio [4]. Además, estos modelos permiten establecer una relación directa entre los requisitos funcionales y los componentes que serán implementados durante el desarrollo del software [3]. Esto favorece la trazabilidad, simplifica la elaboración de pruebas y contribuye al aseguramiento de la calidad del producto final [3]. Los requisitos funcionales deben ser completos, consistentes, verificables y libres de ambigüedades, características que pueden alcanzarse mediante una adecuada utilización de modelos funcionales [1].

2.6. Modelos de Comportamiento: Perspectiva de Comportamiento

2.6.1. Concepto y propósito de los modelos de comportamiento

Los modelos de comportamiento representan el funcionamiento dinámico de un sistema de software, describiendo cómo reaccionan sus componentes ante diferentes eventos y cómo evolucionan a lo largo del tiempo [10]. Mientras que los modelos de datos se enfocan en la información administrada y los modelos funcionales describen las funciones del sistema, los modelos de comportamiento explican la manera en que dichas funciones se ejecutan y cómo interactúan los diferentes elementos durante la operación del software [10]. La especificación de requisitos debe describir no solo las funcionalidades esperadas, sino también el comportamiento del sistema frente a distintas condiciones de operación, garantizando que los requisitos sean completos, verificables y consistentes [1].

En Ingeniería de Software, los modelos de comportamiento son fundamentales porque permiten representar la respuesta del sistema ante acciones de los usuarios, eventos internos o externos y cambios de estado de los objetos [2, 3]. Gracias a estos modelos es posible validar la lógica del sistema antes de iniciar la etapa de implementación, reduciendo errores y facilitando la comunicación entre analistas, diseñadores y desarrolladores [2, 3]. El modelado del comportamiento constituye una práctica esencial durante el análisis y diseño de software, ya que proporciona una visión clara de las interacciones entre los diferentes componentes y facilita la identificación de escenarios excepcionales que podrían afectar el funcionamiento del sistema [15, 16]. El principal objetivo de los modelos de comportamiento es representar la evolución dinámica del sistema durante su ejecución, mostrando cómo responden los objetos y procesos frente a diferentes estímulos o eventos [4]. Estos modelos permiten comprender el flujo temporal de las operaciones y verificar que el comportamiento esperado coincida con los requisitos definidos por los usuarios [4].

2.6.2. Comportamiento dinámico del sistema

El comportamiento dinámico se refiere a la forma en que el sistema cambia de estado en respuesta a estímulos externos o internos [10]. Comprender este comportamiento es crucial para sistemas que manejan múltiples eventos concurrentes o que tienen ciclos de vida complejos [10]. Los modelos de comportamiento permiten describir cómo evoluciona el sistema desde que un usuario inicia una operación hasta que esta finaliza, identificando los eventos que desencadenan determinadas acciones, las respuestas generadas por el software y los cambios de estado que experimentan los diferentes objetos [3].

2.6.3. Diagramas de estados

Uno de los modelos de comportamiento más utilizados es el diagrama de estados, perteneciente a UML [14]. Este diagrama representa los diferentes estados por los que puede pasar un objeto durante su ciclo de vida y las transiciones que ocurren como consecuencia de determinados eventos [14]. Cada estado representa una condición específica del objeto, mientras que las transiciones indican el cambio de un estado a otro cuando se cumple una determinada condición o se produce un evento [10, 15]. Los diagramas de estados permiten identificar claramente el comportamiento de los objetos y verificar que todas las posibles transiciones hayan sido consideradas durante el diseño del sistema, especialmente en aplicaciones con múltiples condiciones de operación [3].

2.6.4. Diagramas de secuencia

Los diagramas de secuencia son otro de los modelos fundamentales de comportamiento definidos por UML [14]. Su finalidad consiste en representar el intercambio de mensajes entre objetos siguiendo un orden cronológico, permitiendo visualizar cómo colaboran los diferentes componentes del sistema para ejecutar una determinada funcionalidad [15]. En este tipo de diagramas participan actores, objetos, líneas de vida, mensajes, activaciones y respuestas [14]. Este tipo de diagramas facilita la comprensión de la lógica interna del sistema y constituye una herramienta muy útil para validar escenarios específicos antes de iniciar la programación [2, 10].

2.6.5. Diagramas de comunicación

Los diagramas de comunicación, anteriormente conocidos como diagramas de colaboración, también forman parte de UML [14] y representan la interacción entre los objetos del sistema desde una perspectiva estructural [14]. A diferencia de los diagramas de secuencia, que priorizan el orden temporal de los mensajes, los diagramas de comunicación destacan las relaciones existentes entre los objetos y la forma en que colaboran para ejecutar una funcionalidad [15]. Cada objeto aparece conectado mediante enlaces de comunicación y los mensajes se identifican mediante una numeración que indica el orden de ejecución [15]. Esta representación resulta útil cuando se desea comprender la arquitectura de colaboración entre componentes sin enfatizar la línea temporal de las operaciones [10].

2.6.6. Importancia de la perspectiva de comportamiento en la validación de requisitos

La perspectiva de comportamiento complementa las perspectivas de datos y funcional, proporcionando una visión dinámica del sistema y permitiendo comprender cómo interactúan los diferentes componentes durante la ejecución de los procesos [4]. Gracias a estos modelos es posible detectar errores de lógica, validar secuencias de operación y mejorar la calidad de la especificación de requisitos [4]. Asimismo, los modelos de comportamiento favorecen la trazabilidad entre los requisitos funcionales y los casos de prueba, ya que describen de forma detallada las respuestas esperadas del sistema frente a diferentes situaciones [3]. Esto facilita la construcción de software más confiable, mantenible y alineado con las necesidades del usuario [3]. La documentación clara del comportamiento del sistema garantiza que los requisitos sean verificables y que puedan utilizarse como base para las actividades de diseño, implementación y pruebas [1].

2.7. Interrelaciones entre las Tres Perspectivas

2.7.1. Relación entre datos, funciones y comportamiento

Durante el proceso de Ingeniería de Requisitos, comprender completamente un sistema de software requiere analizarlo desde diferentes puntos de vista [1]. Cada perspectiva aporta información específica sobre el sistema y, en conjunto, permiten construir una especificación de requisitos completa, consistente y libre de ambigüedades [1]. La documentación de requisitos debe proporcionar múltiples vistas del sistema para facilitar la comunicación entre las partes interesadas y garantizar la trazabilidad durante todo el ciclo de vida del software [1, 4].

Las tres perspectivas más utilizadas son la perspectiva de datos, la perspectiva funcional y la perspectiva de comportamiento [3, 10]. Aunque cada una analiza un aspecto diferente del sistema, ninguna resulta suficiente por sí sola [3, 10]. La integración de estas perspectivas permite comprender no solo qué información administra

el software, sino también qué funciones realiza y cómo se comporta durante su ejecución [3, 10]. La perspectiva de datos se enfoca en la información que será administrada por el sistema, identificando entidades, atributos y relaciones [12]. La perspectiva funcional describe las operaciones, procesos y servicios que el sistema debe proporcionar [5]. La perspectiva de comportamiento describe la forma en que el sistema responde durante su ejecución, incluyendo cambios de estado e interacciones entre objetos [14].

2.7.2. Trazabilidad entre los modelos

La trazabilidad es la capacidad de rastrear los requisitos a lo largo del ciclo de vida del software [2, 4]. La integración de las tres perspectivas facilita la trazabilidad porque cada modelo está vinculado a los demás: los elementos de datos se utilizan en las funciones, y las funciones se ejecutan mediante comportamientos específicos [2, 4]. Los modelos de datos proporcionan la estructura de información que requieren los procesos funcionales; los modelos funcionales definen las operaciones que transforman esos datos; y los modelos de comportamiento especifican la secuencia y las condiciones bajo las cuales se ejecutan dichas operaciones [10].

2.7.3. Consistencia entre las perspectivas

La consistencia entre las perspectivas garantiza que no existan contradicciones entre los modelos [3]. Por ejemplo, los estados definidos en un diagrama de estados deben corresponder a valores posibles de atributos en el modelo de datos, y las operaciones descritas en los casos de uso deben ser coherentes con las transiciones de estado [3]. La verificación de consistencia se realiza mediante revisiones y validaciones cruzadas de los modelos, lo que permite detectar incoherencias antes de la implementación [16].

2.7.4. Ejemplo integrado

A modo de ilustración conceptual, se puede considerar un proceso típico de negocio que involucra la gestión de recursos [2, 10]. Desde la perspectiva de datos, se identifican las entidades y relaciones necesarias para almacenar la información relevante [2, 10]. Desde la funcional, se definen los casos de uso que describen las interacciones de los actores con el sistema para completar el proceso [2, 10]. Desde la de comportamiento, se especifican los diagramas de estados y secuencia que muestran la evolución de los objetos y la colaboración entre componentes durante la ejecución del proceso [2, 10]. Esta integración demuestra que las tres perspectivas representan diferentes niveles de análisis sobre un mismo proceso y que únicamente su utilización conjunta permite obtener una especificación de requisitos completa [2, 10].

2.7.5. Importancia de utilizar las tres perspectivas conjuntamente

La utilización simultánea de las perspectivas de datos, funcional y comportamiento constituye una de las mejores prácticas recomendadas por la Ingeniería de Requisitos moderna [1, 4]. La documentación de requisitos debe proporcionar diferentes vistas del sistema para facilitar la comunicación entre los interesados, mejorar la trazabilidad y reducir la ambigüedad durante el desarrollo [1, 4]. Además, la integración de estas perspectivas aporta múltiples beneficios: mejora la comprensión del dominio del problema, reduce inconsistencias, facilita la comunicación entre clientes y desarrolladores, incrementa la trazabilidad, favorece el diseño de pruebas, disminuye el costo de corrección de errores en etapas posteriores y mejora la calidad del software desarrollado [3, 10]. Una adecuada especificación de requisitos influye directamente en características de calidad como la mantenibilidad, la confiabilidad, la eficiencia y la compatibilidad del software [11].

Las perspectivas de datos, funcional y comportamiento no deben entenderse como modelos independientes, sino como elementos complementarios de una misma especificación de requisitos [3, 10]. Su integración permite construir una representación completa del sistema, proporcionando una base sólida para el diseño, la implementación, las pruebas y el mantenimiento del software [3, 10]. La aplicación conjunta de estas perspectivas, respaldada por normas internacionales como ISO/IEC/IEEE 29148:2018, contribuye significativamente al desarrollo de sistemas de mayor calidad, alineados con las necesidades del usuario y con las buenas prácticas de la Ingeniería de Software [1, 16].

3. Revisión y refinamiento del SRS parcial

3.1. Registro de defectos del SRS parcial

3.2. Corrección de requisitos con sintaxis EARS

3.3. Tabla de atributos de requisitos (RF y RNF)

4. Modelo de datos: entidad-relación y clases UML

4.1. Identificación de entidades y atributos

4.2. Relaciones y cardinalidad

4.3. Diagrama Entidad-Relación (notación Crow's Foot)

4.3.1. Verificación del cumplimiento de la Tercera Forma Normal (3FN)

4.4. Diagrama de clases UML

4.4.1. Diagrama de Clases Conceptual

4.4.2. Justificación de la Refinación Estructural

4.4.3. Diagrama de Clases Refinado

5. Modelo funcional: DFD y diagrama de actividades UML

5.1. Diagrama de Contexto (DFD Nivel 0)

5.2. DFD Nivel 1

5.3. DFD Esencial

5.4. Diagrama de actividades UML

6. Modelo de comportamiento: estados y secuencia

6.1. Selección de la entidad y ciclo de vida

6.2. Diagrama de máquina de estados UML

6.3. Diagrama de secuencia UML

7. Matriz de trazabilidad y tabla comparativa

7.1. Matriz de trazabilidad

7.2. Identificación de gaps y gold plating

7.3. Verificaciones cruzadas entre las tres perspectivas

7.4. Tabla comparativa de los tres tipos de modelos

7.5. Conclusiones sobre la complementariedad de las tres perspectivas

8. Conclusiones

9. Referencias

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering*, ISO/IEC/IEEE 29148:2018(E), Piscataway, NJ, USA, oct. de 2018. DOI: 10.1109/IEEESTD.2018.8559686
- [2] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, Second edition. Berlin, Heidelberg, Germany: Springer-Verlag, 2025, pág. 814, ISBN: 978-3-662-69204-2. DOI: 10.1007/978-3-662-69204-2
- [3] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th. New York, NY, USA: McGraw-Hill Education, 2020, ISBN: 978-1-259-87297-6.
- [4] S. Wagner, D. M. Fernández, M. Kalinowski y M. Felderer, “Agile requirements engineering in practice: Status quo and critical problems,” *CLEI Electronic Journal*, vol. 21, n.º 1, págs. 3-24, 2018, ISSN: 0717-5000. DOI: 10.19153/cleiej.21.1.3
- [5] M. Luckey y G. Engels, “A systematic literature review of use case specifications research,” *Information and Software Technology*, vol. 126, pág. 106346, 2020, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106346
- [6] IREB, *Certified Professional for Requirements Engineering (CPRE) - Foundation Level Syllabus, Version 3.1.0*, Karlsruhe, Germany, 2023. dirección: https://www.ireb.org/en/downloads/_syllabus-foundation-level
- [7] J. Frattini, L. Montgomery, J. Fischbach, D. Mendez, D. Fucci y M. Unterkalmsteiner, “Requirements quality research: a harmonized theory, evaluation, and roadmap,” *Requirements Engineering*, vol. 28, n.º 4, págs. 507-520, 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00405-y
- [8] ISO/IEC/IEEE, *ISO/IEC/IEEE International Standard - Systems and software engineering – System life cycle processes*, ISO/IEC/IEEE 15288:2023(E), Geneva, Switzerland, 2023.
- [9] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz y W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requirements Engineering*, vol. 27, n.º 2, págs. 183-209, 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-021-00367-z
- [10] I. Sommerville, *Software Engineering*, 10th. London, UK: Pearson Higher Education, 2020, ISBN: 978-0-133-94303-0.
- [11] ISO/IEC, *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuARE) – Product quality model*, ISO/IEC 25010:2023(E), Geneva, Switzerland, 2023.
- [12] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7th. Boston, MA, USA: Pearson Education, 2021, ISBN: 978-0-133-97077-7.
- [13] A. Silberschatz, H. F. Korth y S. Sudarshan, “Database System Concepts,” 2020. DOI: 10.1007/978-1-260-08450-4
- [14] Object Management Group, *OMG Unified Modeling Language (OMG UML) Version 2.5.1 Specification*, Formal Specification formal/2017-12-05, Needham, MA, USA, 2017. DOI: 10.1109/ICSE-SEIP.2017.28 dirección: <https://www.omg.org/spec/UML/2.5.1>
- [15] C. F. J. Lange, M. R. V. Chaudron y J. Muskens, “UML model consistency: A systematic literature review,” *Journal of Systems and Software*, vol. 172, pág. 110869, 2021, ISSN: 0164-1212. DOI: 10.1016/j.jss.2020.110869
- [16] H. Washizaki, *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*, H. Washizaki, ed. Los Alamitos, CA, USA: IEEE Computer Society, 2024, ISBN: 979-8-8559-0000-0. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [17] T. Olsson, S. Sentilles y E. Papatheocharous, “A systematic literature review of empirical research on quality requirements,” *Requirements Engineering*, vol. 27, n.º 2, págs. 249-271, 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-022-00373-9
- [18] X. Franch, C. Palomares, C. Quer, P. Chatzipetrou y T. Gorschek, “The state-of-practice in requirements specification: an extended interview study at 12 companies,” *Requirements Engineering*, vol. 28, n.º 3, págs. 377-409, 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00399-7

Anexo A — Tabla de atributos completa de todos los RF y RNF

Anexo B — Registro de defectos del SRS con correcciones aplicadas

Anexo C — Exportaciones originales de todos los diagramas (alta resolución)

Anexo D — Referencias IEEE y declaración de uso de Inteligencia Artificial

Anexo E — Repositorio en GitHub

Enlace al repositorio en GitHub: https://github.com/andymendozap03/IngReq_EquipoB_PEU3_Semana10.git